

This work is our own. _____

This work is our own. _____

Problem 1

Write a program, in either a higher-level programming language or a computer algebra system, to implement the polynomial extended GCD algorithm over the integers mod p . For this program, do not use any built-in functions other than integer addition, subtraction, multiplication, division, the greatest integer function, and the mod p function for p a prime, as well as the usual procedural commands (if/then/else, while, for, and the like).

Use your program to find $d(x)$, the polynomial GCD of $x^{13}+x^8+x^6+x+1$ and x^4+x+1 over the integers mod 2, and to find polynomials $g(x)$ and $h(x)$ mod 2 such that $(x^{13}+x^8+x^6+x+1)g(x)+(x^4+x+1)h(x) \equiv d(x) \pmod{2}$

Listing 1 shows:

$$\begin{aligned}d(x) &= 1 \\g(x) &= x^2 + x \\h(x) &= x^{11} + x^{10} + x^8 + x^3 + 1\end{aligned}$$

Listing 1: Program output for Problem 1.

```
1 p1 = x^13 + x^8 + x^6 + x + 1
2 p2 = x^4 + x + 1
3 d  = 1
4 g  = x^2 + x
5 h  = x^11 + x^10 + x^8 + x^3 + 1
```

We thought about the procedure to generate a finite field given the integers and polynomials. The key is to finding the finite field is to first find the "upper" bound of the finite field we want to generate. In the case of generating finite fields with size of a prime number p , we just need to use p as the upper bound. Since integer mod operation is easy to do, it's fairly simple to have a finite field of size p .

In the case of generating finite field of size p^n , things get more complicated only because the basic arithmetic operations, such as addition, subtraction, multiplication, division, and mod on polynomial are more difficult to do. If we can find ways to do these operations, we are effectively changing the polynomials to an integral domain just like the integers.

With this idea in mind, we create the following classes:

- `IntegralDomain<T>`
- `Int`
- `Polynomial`
- `FiniteField<T>`
- `XGCD<T>`

`IntegralDomain` is an abstract representation for an integral domain. It is intended to unify the interface for the `Int` class and `Polynomial` class, which are the "actual" classes that implements the required arithmetic operations.

Then, the class `FiniteField` will wrap around the two integral domains and create a finite field upon them. The `XGCD` class does the extended GCD algorithm on the two integral domains using the unified interface for basic arithmetic operations.

Please refer to the actual code at the end for more details on our implementation of the basic arithmetic operations and the extended GCD algorithm.

Problem 2

(Stinson, Exercise 3.5) Suppose that we have the following 128-bit AES key, given in hexadecimal notation:

2B7E151628AED2A6ABF7158809CF4F3C

Construct the key schedule arising from this key.

Listing 2: Program output for Problem 2.

1	K0:	2B	7E	15	16	28	AE	D2	A6	AB	F7	15	88	09	CF	4F	3C
2	K1:	A0	FA	FE	17	88	54	2C	B1	23	A3	39	39	2A	6C	76	05
3	K2:	F2	C2	95	F2	7A	96	B9	43	59	35	80	7A	73	59	F6	7F
4	K3:	3D	80	47	7D	47	16	FE	3E	1E	23	7E	44	6D	7A	88	3B
5	K4:	EF	44	A5	41	A8	52	5B	7F	B6	71	25	3B	DB	0B	AD	00
6	K5:	D4	D1	C6	F8	7C	83	9D	87	CA	F2	B8	BC	11	F9	15	BC
7	K6:	6D	88	A3	7A	11	0B	3E	FD	DB	F9	86	41	CA	00	93	FD
8	K7:	4E	54	F7	0E	5F	5F	C9	F3	84	A6	4F	B2	4E	A6	DC	4F
9	K8:	EA	D2	73	21	B5	8D	BA	D2	31	2B	F5	60	7F	8D	29	2F
10	K9:	AC	77	66	F3	19	FA	DC	21	28	D1	29	41	57	5C	00	6E
11	K10:	D0	14	F9	A8	C9	EE	25	89	E1	3F	0C	C8	B6	63	0C	A6

For problem 2, we put the code to generate the round keys inside the **AES** class. We generate all the 40 key columns by the given procedure. Then, augment the columns and build the 10 round key matrices. Lastly, we display the keys in four-byte words. This key expansion is obtained at the beginning of the **encryption** function and used throughout.

Problem 3

Write a program, using either a high-level programming language or a computer algebra system, to implement the AES encryption algorithm described in Section 3.6 of Stinson's text. Use a key length of 128 bits, which translates to $Nr = 10$, where Nr (in Stinson's notation) is the number of rounds. Test your program by encrypting the plaintext consisting entirely of ones, using the complete key schedule you found in Problem 2.

Listing 3 shows the plaintext FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF with the key

2B 7E 15 16 28 AE D2 A6 AB F7 15 88 09 CF 4F 3C encrypts to

8A	F2	86	01	42	F7	86	F4	09	30	7C	1A	3F	7E	AA	AC
----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----

Listing 3: Program output for Problem 3.

```

1 Plain text:      FF FF FF FF  FF FF FF FF  FF FF FF FF  FF FF FF FF
2 Original key:    2B 7E 15 16  28 AE D2 A6  AB F7 15 88  09 CF 4F 3C
3 Adding key...
4   Result:        D4 81 EA E9  D7 51 2D 59  54 08 EA 77  F6 30 B0 C3
5 Round 1
6   Sub Bytes...
7     Result:      48 0C 87 1E  0E D1 D8 CB  20 30 87 F5  42 04 E7 2E
8   Shift Rows...
9     Result:      48 D1 87 2E  0E 30 E7 1E  20 04 87 CB  42 0C D8 F5
10  Mixing cols...
11    Result:      51 4D FE D2  B5 42 C9 F9  00 71 77 6E  BD DC E1 E3
12  Add round key 1...
13  Round key 1: A0 FA FE 17  88 54 2C B1  23 A3 39 39  2A 6C 76 05
14    Result:      F1 B7 00 C5  3D 16 E5 48  23 D2 4E 57  97 B0 97 E6
15 Round 2
16  Sub Bytes...
17    Result:      A1 A9 63 A6  27 47 D9 52  26 B5 2F 5B  88 E7 88 8E
18  Shift Rows...
19    Result:      A1 47 2F 8E  27 B5 88 A6  26 E7 63 52  88 A9 D9 5B
20  Mixing cols...
21    Result:      31 D0 31 97  A4 73 68 03  4F 04 F1 4A  69 EA 65 45
22  Add round key 2...
23  Round key 2: F2 C2 95 F2  7A 96 B9 43  59 35 80 7A  73 59 F6 7F
24    Result:      C3 12 A4 65  DE E5 D1 40  16 31 71 30  1A B3 93 3A
25 Round 3
26  Sub Bytes...
27    Result:      2E C9 49 4D  1D D9 3E 09  47 C7 A3 04  A2 6D DC 80
28  Shift Rows...
29    Result:      2E D9 A3 80  1D C7 DC 4D  47 6D 49 09  A2 C9 3E 04
30  Mixing cols...
31    Result:      0F F9 31 13  F9 BA AE A6  79 4F A3 FF  25 6D 1B 02
32  Add round key 3...
33  Round key 3: 3D 80 47 7D  47 16 FE 3E  1E 23 7E 44  6D 7A 88 3B
34    Result:      32 79 76 6E  BE AC 50 98  67 6C DD BB  48 17 93 39
35 Round 4
36  Sub Bytes...
37    Result:      23 B6 38 9F  AE 91 53 46  85 50 C1 EA  52 F0 DC 12
38  Shift Rows...
39    Result:      23 91 C1 12  AE 50 DC 9F  85 F0 38 46  52 B6 53 EA
40  Mixing cols...
41    Result:      3D 50 1D 11  F4 EE E7 40  64 70 CF D0  DC 3A 67 DC
42  Add round key 4...
43  Round key 4: EF 44 A5 41  A8 52 5B 7F  B6 71 25 3B  DB 0B AD 00
44    Result:      D2 14 B8 50  5C BC BC 3F  D2 01 EA EB  07 31 CA DC
45 Round 5
46  Sub Bytes...
47    Result:      B5 FA 6C 53  4A 65 65 75  B5 7C 87 E9  C5 C7 74 86
48  Shift Rows...
49    Result:      B5 65 87 86  4A 7C 74 53  B5 C7 6C 75  C5 FA 65 E9
50  Mixing cols...
51    Result:      DF 6B 54 31  37 7D 2B 70  3A E1 35 85  08 6C D5 02
52  Add round key 5...
53  Round key 5: D4 D1 C6 F8  7C 83 9D 87  CA F2 B8 BC  11 F9 15 BC
54    Result:      0B BA 92 C9  4B FE B6 F7  F0 13 8D 39  19 95 C0 BE

```

```

55 Round 6
56   Sub Bytes...
57     Result:  2B F4 4F DD  B3 BB 4E 68  8C 7D 5D 12  D4 2A BA AE
58   Shift Rows...
59     Result:  2B BB 5D AE  B3 7D BA DD  8C 2A 4F 68  D4 F4 4E 12
60   Mixing cols...
61     Result:  73 0F C3 DC  9D 41 DD A8  5A 61 80 3A  E8 E7 8A F9
62   Add round key 6...
63   Round key 6: 6D 88 A3 7A  11 0B 3E FD  DB F9 86 41  CA 00 93 FD
64     Result:  1E 87 60 A6  8C 4A E3 55  81 98 06 7B  22 E7 19 04
65 Round 7
66   Sub Bytes...
67     Result:  72 17 D0 24  64 D6 11 FC  0C 46 6F 21  93 94 D4 F2
68   Shift Rows...
69     Result:  72 D6 6F F2  64 46 D4 24  0C 94 D0 FC  93 17 11 21
70   Mixing cols...
71     Result:  18 86 77 D0  F2 AB FD 76  93 A8 3C B3  34 AF C5 EA
72   Add round key 7...
73   Round key 7: 4E 54 F7 0E  5F 5F C9 F3  84 A6 4F B2  4E A6 DC 4F
74     Result:  56 D2 80 DE  AD F4 34 85  17 0E 73 01  7A 09 19 A5
75 Round 8
76   Sub Bytes...
77     Result:  B1 B5 CD 1D  95 BF 18 97  F0 AB 8F 7C  DA 01 D4 06
78   Shift Rows...
79     Result:  B1 BF 8F 06  95 AB D4 1D  F0 01 CD 97  DA B5 18 7C
80   Mixing cols...
81     Result:  2A 58 01 F4  1E A2 AA E1  A2 29 D2 F2  0F FF DB 20
82   Add round key 8...
83   Round key 8: EA D2 73 21  B5 8D BA D2  31 2B F5 60  7F 8D 29 2F
84     Result:  C0 8A 72 D5  AB 2F 10 33  93 02 27 92  70 72 F2 0F
85 Round 9
86   Sub Bytes...
87     Result:  BA 7E 40 03  62 15 CA C3  DC 77 CC 4F  51 40 89 76
88   Shift Rows...
89     Result:  BA 15 CC 76  62 77 89 03  DC 40 40 C3  51 7E CA 4F
90   Mixing cols...
91     Result:  EA A9 B6 E0  D7 0F 19 5E  E0 5F 42 E2  A5 A7 71 D9
92   Add round key 9...
93   Round key 9: AC 77 66 F3  19 FA DC 21  28 D1 29 41  57 5C 00 6E
94     Result:  46 DE D0 13  CE F5 C5 7F  C8 8E 6B A3  F2 FB 71 B7
95 Round 10
96   Sub Bytes...
97     Result:  5A 1D 70 7D  8B E6 A6 D2  E8 19 7F 0A  89 0F A3 A9
98   Shift Rows...
99     Result:  5A E6 7F A9  8B 19 A3 7D  E8 0F 70 D2  89 1D A6 0A
100   Add round key 10...
101   Round key 10: D0 14 F9 A8  C9 EE 25 89  E1 3F 0C C8  B6 63 0C A6
102     Result:  8A F2 86 01  42 F7 86 F4  09 30 7C 1A  3F 7E AA AC

```

We create a main `encrypt()` function to coordinate between the four independent routines for AES encryption. Both the `encrypt()` function and the other routine functions are in the `AES` class.

Please refer to the code below for details of the implementation.

Source Code

Listing 4: MainClass.java

```

1 import java.util.List;
2
3 /**
4  * MainClass.java
5  * The main entry point of the code for Problem Set 11.
6  *
7  * @author Brandon Amos, Tianyu Geng
8  * @version Dec 2, 2012
9  */
10 public class MainClass {
11     public static void main(String[] args) {
12         H.msg("Problem 1:");
13         // Set up the upper bound of the coefficients
14         Int N = new Int(2);
15         // Set up the zero coefficients.
16         FiniteField<Int> zero = new FiniteField<Int>(N);
17
18         Polynomial p1 = new Polynomial(zero);
19         p1.setCoefficients(1, 1, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 1);
20         System.out.println("p1 = " + p1);
21         Polynomial p2 = new Polynomial(zero);
22         p2.setCoefficients(1, 1, 0, 0, 1);
23         System.out.println("p2 = " + p2);
24         // Carry out the extended GCD algorithm.
25         XGCD<Polynomial> xgcd = new XGCD<Polynomial>();
26         List<Polynomial> results = xgcd.xgcd(p1, p2);
27         Polynomial d = results.get(0);
28         Polynomial g = results.get(1);
29         Polynomial h = results.get(2);
30         System.out.println("d = " + d);
31         System.out.println("g = " + g);
32         System.out.println("h = " + h);
33
34         H.line();
35         H.msg("Problem 2:");
36         int[][] key =
37             { { 0x2b, 0x7e, 0x15, 0x16 }, { 0x28, 0xae, 0xd2, 0xa6 },
38               { 0xab, 0xf7, 0x15, 0x88 },
39               { 0x09, 0xcf, 0x4f, 0x3c } };
40         int[][] state =
41             { { 0xFF, 0xFF, 0xFF, 0xFF }, { 0xFF, 0xFF, 0xFF, 0xFF },
42               { 0xFF, 0xFF, 0xFF, 0xFF },
43               { 0xFF, 0xFF, 0xFF, 0xFF } };
44         AES aes = new AES(state, key);
45         aes.printKeyExpansion();
46
47         H.line();
48         H.msg("Problem 3:");
49         aes.encrypt();
50     }
51 }

```

Listing 5: AES.java

```

1 import java.util.Arrays;
2
3 /**
4  * AES.java
5  * Implements the key generation and encryption for AES
6  *
7  * @author Brandon Amos, Tianyu Geng
8  * @version Dec 2, 2012
9  */
10 public class AES {
11     private int [][] state;
12     private int [][] key;
13     private RoundKey[] roundKeys;
14
15     /**
16      * @param state The initial state. For encryption, this is the plaintext.
17      */
18     public AES(int [][] state, int [][] key) {
19         this.state = state;
20         this.key = key;
21         roundKeys = keyExpansion(key);
22     }
23
24     /**
25      * Print the key expansion
26      */
27     public void printKeyExpansion() {
28         RoundKey[] roundKeys = keyExpansion(key);
29
30         for (int i = 0; i < roundKeys.length; i++) {
31             H.msg("K" + i + ":\t" + H.matrixToString(roundKeys[i].keyBytes));
32         }
33     }
34
35     /**
36      * Encryption method, printing each step.
37      */
38     public void encrypt() {
39         System.out.print("Plain text:      ");
40         H.display(state);
41         System.out.print("Original key:      ");
42         H.display(roundKeys[0].keyBytes);
43         System.out.println("Adding key ... ");
44         addKey(roundKeys[0]);
45         System.out.print("      Result:      ");
46         H.display(state);
47
48         for (int i = 1; i <= 9; i++) {
49             System.out.println("Round " + i);
50             System.out.println("      Sub Bytes ... ");
51             subBytes();
52             System.out.print("      Result:      ");
53             H.display(state);
54             System.out.println("      Shift Rows...");

```

```

55         shiftRows();
56         System.out.print("          Result:  ");
57         H.display(state);
58         System.out.println("      Mixing cols...");
59         mixColumn();
60         System.out.print("          Result:  ");
61         H.display(state);
62         System.out.println("      Add round key " + i + "...");
63         System.out.print("      Round key " + i + ": ");
64         H.display(roundKeys[i].keyBytes);
65
66         addKey(roundKeys[i]);
67         System.out.print("          Result:  ");
68         H.display(state);
69     }
70     System.out.println("Round 10");
71     System.out.println("      Sub Bytes... ");
72     subBytes();
73     System.out.print("          Result:  ");
74     H.display(state);
75     System.out.println("      Shift Rows...");
76     shiftRows();
77     System.out.print("          Result:  ");
78     H.display(state);
79     System.out.println("      Add round key 10...");
80     System.out.print("      Round key 10:");
81     H.display(roundKeys[10].keyBytes);
82     addKey(roundKeys[10]);
83     System.out.print("          Result:  ");
84     H.display(state);
85
86 }
87
88 /**
89  * XOR the state matrix by the round key
90  * @param key The round key
91  */
92 private void addKey(RoundKey key) {
93     state = H.XOR(key.keyBytes, state);
94 }
95
96 /**
97  * Calculate the round keys from the initial key
98  * @param key
99  * @return an array of round keys
100 */
101 private RoundKey[] keyExpansion(int[][] key) {
102     KeyCol[] W = new KeyCol[44];
103
104     // Initialize it with the key
105     for (int i = 0; i < 4; i++) {
106         W[i] = new KeyCol();
107         W[i].colBytes = key[i];
108     }
109

```

```

110     for (int i = 4; i <= 43; i++) {
111         W[i] = new KeyCol();
112         if (i % 4 != 0) {
113             W[i].colBytes =
114                 H.XOR(W[i - 1].colBytes, W[i - 4].colBytes);
115         }
116         else {
117             int[] W1Prev = W[i - 1].colBytes;
118             int[] W4Prev = W[i - 4].colBytes;
119             PolyByte x = new PolyByte(2);
120             x = x.pow((i - 4) / 4);
121             int[] rhs =
122                 { subByteOperation(W1Prev[1]) ^ x.toByte(),
123                   subByteOperation(W1Prev[2]),
124                   subByteOperation(W1Prev[3]),
125                   subByteOperation(W1Prev[0]) };
126             W[i].colBytes = H.XOR(W4Prev, rhs);
127         }
128     }
129
130     return getKeysFromW(W);
131 }
132
133 /**
134  * From W, extract the round keys.
135  * @param W The calculated columns from keyExpansion
136  * @return The round keys
137  */
138 private RoundKey[] getKeysFromW(KeyCol[] W) {
139     int numKeys = W.length / 4;
140     RoundKey[] keys = new RoundKey[numKeys];
141     for (int i = 0; i < numKeys; i++) {
142         keys[i] = new RoundKey();
143         for (int j = 0; j < 4; j++) {
144             for (int k = 0; k < 4; k++) {
145                 keys[i].keyBytes[j][k] = W[4 * i + k].colBytes[j];
146             }
147         }
148     }
149     return keys;
150 }
151
152 /**
153  * Substitute the bytes in the entire state matrix
154  */
155 private void subBytes() {
156     for (int i = 0; i < 4; i++) {
157         for (int j = 0; j < 4; j++) {
158             state[i][j] = subByteOperation(state[i][j]);
159         }
160     }
161 }
162
163 /**
164  * Substitute a byte.

```



```

165     * @return The byte
166     */
167     private int subByteOperation(int inInt) {
168         int z = 0;
169         PolyByte y = new PolyByte(inInt);
170         z = y.inverse().toByte();
171         z = subByteMult(z);
172         return z;
173     }
174
175     /**
176     * Perform the matrix multiplication when substituting a byte.
177     * @param inByte The byte to multiply
178     * @return The result
179     */
180     private int subByteMult(int inByte) {
181         int[] outBits = new int[8];
182         int[] bits = H.byteToReversedBits(inByte);
183         int[] mult = { 1, 0, 0, 0, 1, 1, 1, 1 };
184         int[] add = { 1, 1, 0, 0, 0, 1, 1, 0 };
185         for (int i = 0; i < 8; i++) {
186             int tmpSum = 0;
187             for (int j = 0; j < 8; j++) {
188                 int multBit = mult[H.posMod(j - i, 8)];
189                 tmpSum += multBit * bits[j];
190             }
191             outBits[i] = (tmpSum + add[i]) % 2;
192         }
193         return H.reverseBitsToByte(outBits);
194     }
195
196     /**
197     * Shift the rows of the state matrix.
198     */
199     private void shiftRows() {
200         int[][] oldState = new int[state.length][state.length];
201
202         for (int i = 0; i < state.length; i++) {
203             for (int j = 0; j < state.length; j++) {
204                 oldState[i][j] = state[i][j];
205             }
206         }
207
208         for (int i = 0; i < 4; i++) {
209             for (int j = 0; j < 4; j++) {
210                 state[i][j] = oldState[i][(i + j) % 4];
211             }
212         }
213     }
214
215     /**
216     * Generate the matrix M for column mixing.
217     */
218     static private PolyByte[][] M = new PolyByte[4][4];
219     {

```

```

220     M[0][0] = new PolyByte(2);
221     M[0][1] = new PolyByte(3);
222     M[0][2] = new PolyByte(1);
223     M[0][3] = new PolyByte(1);
224     M[1][0] = new PolyByte(1);
225     M[1][1] = new PolyByte(2);
226     M[1][2] = new PolyByte(3);
227     M[1][3] = new PolyByte(1);
228     M[2][0] = new PolyByte(1);
229     M[2][1] = new PolyByte(1);
230     M[2][2] = new PolyByte(2);
231     M[2][3] = new PolyByte(3);
232     M[3][0] = new PolyByte(3);
233     M[3][1] = new PolyByte(1);
234     M[3][2] = new PolyByte(1);
235     M[3][3] = new PolyByte(2);
236
237 }
238
239 /**
240  * Mix the columns
241  */
242 private void mixColumn() {
243     // create a 2-D array to hold the PolyBytes for calculation.
244     PolyByte[][] statepb = new PolyByte[4][4];
245     // Do the matrix multiplication
246     for (int i = 0; i < 4; i++) {
247         for (int j = 0; j < 4; j++) {
248             statepb[i][j] = new PolyByte(0);
249             for (int k = 0; k < 4; k++) {
250                 statepb[i][j] =
251                     statepb[i][j].add(M[i][k].mul(new PolyByte(
252                         state[k][j]))));
253             }
254         }
255     }
256     // convert from PolyByte to integer byte.
257     for (int i = 0; i < 4; i++) {
258         for (int j = 0; j < 4; j++) {
259             state[i][j] = statepb[i][j].toByte();
260         }
261     }
262 }
263
264 /**
265  * A class that wraps a 2d array to hold the round keys in.
266  * Used for readability.
267  */
268 private class RoundKey {
269     public int[][] keyBytes;
270
271     public RoundKey() {
272         keyBytes = new int[4][4];
273     }
274

```

```

275     }
276
277     /**
278      * A class that wraps an array to hold key columns in.
279      * Used for readability.
280      */
281     private class KeyCol {
282         public int [] colBytes;
283
284         public String toString() {
285             return Arrays.toString(colBytes);
286         }
287     }
288 }

```

Listing 6: H.java

```

1  /**
2   * H.java
3   * Provide helper functionalities.
4   *
5   * @author Brandon Amos, Tianyu Geng
6   * @version Dec 2, 2012
7   */
8  public class H {
9      public static void msg(String message) {
10         System.out.println(message);
11     }
12
13     public static void line() {
14         System.out.println();
15     }
16
17     /**
18      * Print a byte as a 2-character hexadecimal digit
19      */
20     public static String byteHexStr(int message) {
21         return String.format("%02X", message);
22     }
23
24     /**
25      * Convert a byte to an array of bits.
26      * @return the array of bits.
27      */
28     public static int [] byteToBits(int inByte) {
29         int [] bits = new int [8];
30         int mask = 0x80;
31         for (int i = 0; i < 8; i++) {
32             bits[i] = (inByte & mask) != 0 ? 1 : 0;
33             mask >>= 1;
34         }
35         return bits;
36     }
37
38     /**

```

```

39  * Convert a byte into a reversed array of bits.
40  * @return the reversed array of bits.
41  */
42  public static int[] byteToReversedBits(int inByte) {
43      int[] bits = new int[8];
44      int mask = 1;
45      for (int i = 0; i < 8; i++) {
46          bits[i] = (inByte & mask) != 0 ? 1 : 0;
47          mask <<= 1;
48      }
49      return bits;
50  }
51
52  /**
53   * Convert an array of bits to a byte.
54   * @return the byte
55   */
56  public static int bitsToByte(int[] bits) {
57      int byteVal = 0;
58      for (int i = 0; i < 8; i++) {
59          byteVal <<= 1;
60          byteVal |= bits[i];
61      }
62      return byteVal;
63  }
64
65  /**
66   * Convert an array of reversed bits to a byte.
67   * @return the byte
68   */
69  public static int reverseBitsToByte(int[] bits) {
70      int byteVal = 0;
71      for (int i = 7; i >= 0; i--) {
72          byteVal <<= 1;
73          byteVal |= bits[i];
74      }
75      return byteVal;
76  }
77
78  /**
79   * Display a 2d matrix
80   */
81  public static void display(int[][] matrix) {
82      System.out.println(matrixToString(matrix));
83  }
84
85  public static String matrixToString(int[][] matrix) {
86      StringBuilder b = new StringBuilder();
87      for (int i = 0; i < 4; i++) {
88          for (int j = 0; j < 4; j++) {
89              b.append(H.byteHexStr(matrix[j][i]) + " ");
90          }
91          b.append("\n");
92      }
93      return b.toString();

```

```
94     }
95
96     /**
97      * @return the positive modulus of 2 integers
98      */
99     public static int posMod(int rhs, int lhs) {
100         return (rhs % lhs + lhs) % lhs;
101     }
102
103     /**
104      * @return the XOR of the elements of the 2d matrices
105      */
106     public static int [][] XOR(int [][] lhs, int [][] rhs) {
107         int [][] out = new int[lhs.length][lhs.length];
108         for (int i = 0; i < lhs.length; i++) {
109             for (int j = 0; j < lhs.length; j++) {
110                 out[i][j] = lhs[i][j] ^ rhs[i][j];
111             }
112         }
113         return out;
114     }
115
116
117     /**
118      * @return the XOR of the elements of the matrices
119      */
120     public static int [] XOR(int [] lhs, int [] rhs) {
121         int [] out = new int[lhs.length];
122         for (int i = 0; i < lhs.length; i++) {
123             out[i] = lhs[i] ^ rhs[i];
124         }
125         return out;
126     }
127 }
```

Listing 7: IntegralDomain.java

```

1 /**
2  * IntegralDomain.java
3  * This class is an abstraction for a mathematical integral domain.
4  *
5  * @author Tianyu Geng, Brandon Amos
6  * @version Dec 2, 2012
7  * @param <T>
8  *     The type that will implement this interface
9  */
10 public interface IntegralDomain<T> {
11
12     /**
13      * @return the zero in this integral domain.
14      */
15     public abstract T zero();
16
17     /**
18      * @return the one in this integral domain.
19      */
20     public abstract T one();
21
22     /**
23      * Determine whether two integral domain are equal or not.
24      *
25      * @param i
26      *     the other integral domain
27      * @return Return true if they equal; otherwise, false.
28      */
29     public abstract boolean equal(T i);
30
31     /**
32      * Add this integral domain with another integral domain and return the
33      * result.
34      *
35      * @param i
36      *     the other integral domain
37      * @return the result
38      */
39     public abstract T add(T i);
40
41     /**
42      * Multiply this integral domain with another integral domain and return the
43      * result.
44      *
45      * @param i
46      *     the other integral domain
47      * @return the result
48      */
49     public abstract T mul(T i);
50
51     /**
52      * Subtract this integral domain by another integral domain and return the
53      * result.
54      *

```

```

55     * @param i
56     *         the other integral domain
57     * @return the result
58     */
59     public abstract T sub(T i);
60
61     /**
62     * Divide this integral domain by another integral domain and return the
63     * result.
64     *
65     * @param i
66     *         the other integral domain
67     * @return the result
68     */
69     public abstract T div(T i);
70
71     /**
72     * Mod this integral domain by another integral domain and return the
73     * result.
74     *
75     * @param i
76     *         the other integral domain
77     * @return the result
78     */
79     public abstract T mod(T i);
80
81     /**
82     * Find the additive inverse of this integral domain return the result.
83     *
84     * @return the result
85     */
86     public abstract T minus();
87
88     /**
89     * Set the value of this integral domain.
90     *
91     * @param i
92     */
93     public abstract void setValue(T i);
94
95 }

```

Listing 8: XGCD.java

```

1 import java.util.ArrayList;
2
3 /**
4  * XGCD.java
5  * Implement the extended GCD algorithm on integral domain T.
6  *
7  * @author Tianyu Geng, Brandon Amos
8  * @version Dec 2, 2012
9  * @param <T>
10  *         the integral domain to carry out this algorithm
11  */

```

```

12 public class XGCD<T> extends IntegralDomain<T>> {
13     /**
14      * This function will do the algorithm and return a list of integral domains
15      * which are the gcd, p, and q, where
16      *
17      *  $m \cdot p + n \cdot q = d$ 
18      *
19      * @param m
20      *         the first integral domain
21      * @param n
22      *         the second integral domain
23      * @return the list containing gcd, p, and q. Note you can do .get(0) to get
24      *         the gcd; .get(1) to get p, and .get(2) to get q.
25      */
26     public ArrayList<T> xgcd(T m, T n) {
27         ArrayList<T> rpq = new ArrayList<T>(3); // The result list
28
29         T temp;
30         // The list of quotients, used to find p and q at the end
31         ArrayList<T> quotients = new ArrayList<T>();
32         // Carry out the GCD algorithm
33         while (!(m.equal(m.zero()))) {
34             quotients.add(n.div(m));
35             temp = m;
36             m = n.mod(m);
37             n = temp;
38         }
39         // Put the gcd into the result list.
40         rpq.add(n);
41         // Begin to find p and q. We will use the 'magic' Dr. Brown showed in
42         // class for the calculation.
43
44         // First put zero and one at the end of the array.
45         quotients.add(m.zero());
46         quotients.set(quotients.size() - 2, m.one());
47         // Then do the magic calculation.
48         for (int i = quotients.size() - 3; i >= 0; i--) {
49             quotients.set(i, quotients.get(i).mul(quotients.get(i + 1))
50                 .add(quotients.get(i + 2)));
51         }
52         // Put the result into the result list with the correct sign.
53         if (quotients.size() % 2 == 0) {
54             rpq.add(quotients.get(0));
55             rpq.add(quotients.get(1).minus());
56         }
57         else {
58             rpq.add(quotients.get(0).minus());
59             rpq.add(quotients.get(1));
60         }
61
62         return rpq;
63     }
64 }
65 }

```


Listing 9: Int.java

```
1 /**
2  * Int.java
3  * This class represents an integer just like the Java's wrapper class for int.
4  * This is used to track the zero and one in the integral domain.
5  * This also gives functionality to FiniteField to build a finite field on
6  * an integral domain.
7  *
8  * @author Tianyu Geng, Brandon Amos
9  * @version Dec 2, 2012
10 */
11 public class Int implements IntegralDomain<Int> {
12     private int value;
13
14     /**
15      * Constructor for Int
16      *
17      * @param v
18      *         the value for this int.
19      */
20     public Int(int v) {
21         value = v;
22     }
23
24     /**
25      * Get the integer value.
26      *
27      * @return the value of this Int in int.
28      */
29     public int getValue() {
30         return value;
31     }
32
33     @Override
34     public Int zero() {
35         return new Int(0);
36     }
37
38     @Override
39     public Int one() {
40         return new Int(1);
41     }
42
43     @Override
44     public boolean equal(Int i) {
45         return value == i.value;
46     }
47
48     @Override
49     public Int add(Int i) {
50         return new Int(value + i.value);
51     }
52
53     @Override
54     public Int mul(Int i) {
```

```

55     return new Int(value * i.value);
56 }
57
58 @Override
59 public Int sub(Int i) {
60     return new Int(value - i.value);
61 }
62
63 @Override
64 public Int div(Int i) {
65     return new Int(value / i.value);
66 }
67
68 @Override
69 public Int mod(Int i) {
70     return new Int((value % i.value + i.value) % i.value);
71 }
72
73 @Override
74 public Int minus() {
75     return new Int(-value);
76 }
77
78 /**
79  * Set the value for this Int
80  *
81  * @param i
82  *         the new value for this int.
83  */
84 public void setValue(Int i) {
85     this.value = i.value;
86 }
87
88 @Override
89 public String toString() {
90     return Integer.toString(value);
91 }
92
93 }

```

Listing 10: FiniteField.java

```

1 /**
2  * FiniteField.java
3  * This is an abstraction of a finite field basing on an integral domain. It
4  * uses the extended GCD algorithm to find the inverse.
5  *
6  * @author Tianyu Geng, Brandon Amos
7  * @version Dec 2, 2012
8  * @param <T>
9  *         The type of integral domain this finite field should be built upon.
10 */
11 public class FiniteField<T extends IntegralDomain<T>> {
12
13     /**

```

```

14     * The "upper" bound (a prime integer or an irreducible polynomial)
15     */
16     public final T N;
17
18     private T value;
19
20     /**
21     * Constructor for FiniteField.
22     *
23     * @param value
24     *         the internal value representing this finite field
25     * @param N
26     *         the upper bound
27     */
28     public FiniteField(T value, T N) {
29         this.N = N;
30         this.value = value.mod(N);
31     }
32
33     /**
34     * Constructor for zero FiniteField.
35     *
36     * @param N
37     *         the upper bound
38     */
39     public FiniteField(T N) {
40         this.N = N;
41         this.value = N.zero();
42     }
43
44     /**
45     * @return the value of the integral domain.
46     */
47     public T getValue() {
48         return value;
49     }
50
51     /**
52     * @return zero of this finite field
53     */
54     public FiniteField<T> zero() {
55         return new FiniteField<T>(N);
56     }
57
58     /**
59     * @return one of this finite field
60     */
61
62     public FiniteField<T> one() {
63         return new FiniteField<T>(N.one(), N);
64     }
65
66     /**
67     * Determine whether two finite fields are equal. Note, comparing the
68     * internal integral domain is enough because the finite field will always

```

```

69      * be created with the internal value smaller than the upper bound.
70      *
71      * @param f the
72      *         other finite field
73      * @return Return true if two finite field are equal; otherwise false.
74      */
75
76      public boolean equal(FiniteField<T> f) {
77
78          return value.equal(f.value);
79      }
80
81      /**
82       * Add this finite field with another finite field and return the result
83       *
84       * @param f
85       *         the other finite field
86       * @return the result
87       */
88      public FiniteField<T> add(FiniteField<T> f) {
89          return new FiniteField<T>(value.add(f.value), N);
90      }
91
92      /**
93       * Multiply this finite field with another finite field and return the
94       * result
95       *
96       * @param f
97       *         the other finite field
98       * @return the result
99       */
100     public FiniteField<T> mul(FiniteField<T> f) {
101         return new FiniteField<T>(value.mul(f.value), N);
102     }
103
104     /**
105      * Subtract this finite field by another finite field and return the result
106      *
107      * @param f
108      *         the other finite field
109      * @return the result
110      */
111     public FiniteField<T> sub(FiniteField<T> f) {
112         return new FiniteField<T>(value.sub(f.value), N);
113     }
114
115     /**
116      * Divide this finite field by another finite field and return the result
117      *
118      * @param f
119      *         the other finite field
120      * @return the result
121      */
122     public FiniteField<T> div(FiniteField<T> f) {
123         return this.mul(f.inverse());

```

```

124     }
125
126     /**
127      * Get the additive inverse of this finite field.
128      *
129      * @return the additive inverse
130      */
131     public FiniteField<T> minus() {
132         return new FiniteField<T>(value.minus(), N);
133     }
134
135     /**
136      * Get the multiplicative inverse of this finite field.
137      *
138      * @return the multiplicative inverse
139      */
140     public FiniteField<T> inverse() {
141         // Create a XGCD object to do the extended GCD algorithm and find the
142         // inverse
143         XGCD<T> xgcd = new XGCD<T>();
144
145         try {
146             return new FiniteField<T>(xgcd.xgcd(value, N).get(1), N);
147         }
148         // In the case of a polynomial, when the current polynomial is zero, it's
149         // length will be 0 and the following exception will be thrown.
150         catch (ArrayIndexOutOfBoundsException e) {
151
152             throw new ArithmeticException("Division by zero: "
153                 + this.toString());
154         }
155     }
156
157     @Override
158     public String toString() {
159         return value.toString();
160     }
161
162 }

```

Listing 11: Polynomial.java

```

1 import java.util.List;
2 import java.util.ArrayList;
3
4 /**
5  * Polynomial.java
6  * This class represents a polynomial with coefficients of a finite field on
7  * integers. It support a minimal set of arithmetic operations.
8  *
9  * @author Tianyu Geng, Brandon Amos
10 * @version Dec 2, 2012
11 */
12 public class Polynomial implements IntegralDomain<Polynomial> {
13     // The zero of its coefficient.
14     private FiniteField<Int> zero;
15     // The list of coefficients in this polynomial.
16     private List<FiniteField<Int>> coefficients;
17
18     /**
19      * Constructor for Polynomial
20      *
21      * @param zero
22      * the zero elements in that finite field. This is necessary
23      * because this polynomial supports finite field with size of any
24      * prime number. This zero will contain the this very
25      * information.
26      */
27     public Polynomial(FiniteField<Int> zero) {
28         this.zero = zero;
29         coefficients = new ArrayList<FiniteField<Int>>();
30     }
31
32     /**
33      * Constructor a copy of Polynomial p.
34      *
35      * @param p
36      * the original polynomial
37      */
38     public Polynomial(Polynomial p) {
39         coefficients = new ArrayList<FiniteField<Int>>(p.coefficients);
40         this.zero = p.zero;
41     }
42
43     /**
44      * Set/reset the coefficients of this polynomial. (Note if the integer is
45      * bigger than the prime number chosen to create the finite field, the mod
46      * operation will be carried out during construction of the finite field)
47      *
48      * @param ints
49      * the coefficients
50      */
51     public void setCoefficients(int... ints) {
52         coefficients.clear();
53         for (int i : ints) {

```

```

55         coefficients.add(new FiniteField<Int>(new Int(i), zero.N));
56     }
57     this.clean();
58 }
59
60 /**
61  * @return the list of coefficients.
62  */
63 public List<FiniteField<Int>> getCoefficients() {
64     return this.coefficients;
65 }
66
67 /**
68  * Add a higher order element to the polynomial with coefficient c. This
69  * method is intended to use to build the polynomial term by term.
70  *
71  * @param c
72  *         the coefficient for the next high order.
73  * @return Return the order of this polynomial after this operation
74  */
75 public int addHigherPower(FiniteField<Int> c) {
76     this.coefficients.add(c);
77     return this.coefficients.size() - 1;
78 }
79
80 @Override
81 public Polynomial zero() {
82     return new Polynomial(zero);
83 }
84
85 @Override
86 public Polynomial one() {
87     Polynomial one = new Polynomial(zero);
88     one.coefficients.add(zero.one());
89     return one;
90 }
91
92 @Override
93 public boolean equal(Polynomial i) {
94     if (coefficients.size() != i.coefficients.size())
95         return false;
96     // compare the coefficients
97     for (int j = 0; j < coefficients.size(); j++) {
98         if (!coefficients.get(j).equal(i.coefficients.get(j))) {
99             return false;
100         }
101     }
102     return true;
103 }
104
105 @Override
106 public Polynomial add(Polynomial p) {
107     Polynomial result = new Polynomial(zero);
108     // Add the coefficients piece by piece.
109     if (this.coefficients.size() < p.coefficients.size()) {

```

```

110         int i;
111         for (i = 0; i < this.coefficients.size(); i++) {
112             result.coefficients.add(this.coefficients.get(i).add(
113                 p.coefficients.get(i)));
114         }
115         for (; i < p.coefficients.size(); i++) {
116             result.coefficients.add(p.coefficients.get(i));
117         }
118     }
119     else {
120         int i;
121         for (i = 0; i < p.coefficients.size(); i++) {
122             result.coefficients.add(this.coefficients.get(i).add(
123                 p.coefficients.get(i)));
124         }
125         for (; i < this.coefficients.size(); i++) {
126             result.coefficients.add(this.coefficients.get(i));
127         }
128     }
129     result.clean();
130     return result;
131 }
132
133 @Override
134 public Polynomial mul(Polynomial p) {
135     Polynomial result = new Polynomial(zero);
136     // Create a polynomial to hold the result and put zeros to the terms.
137     for (int i = 0; i < this.coefficients.size()
138         + p.coefficients.size() - 1; i++) {
139         result.coefficients.add(zero);
140     }
141     // For each term in the current polynomial and p, calculate the product
142     // of the coefficient and add this result to the overall result.
143     for (int i = 0; i < this.coefficients.size(); i++) {
144         for (int j = 0; j < p.coefficients.size(); j++) {
145             result.coefficients.set(
146                 i + j,
147                 result.coefficients.get(i + j).add(
148                     this.coefficients.get(i).mul(
149                         p.coefficients.get(j))));
150         }
151     }
152     result.clean();
153     return result;
154 }
155
156 /**
157  * Clean up the high order terms with zero coefficients.
158  *
159  * @return the number of terms cleaned.
160  */
161 public int clean() {
162     int counter = 0;
163     while (!this.coefficients.isEmpty()
164         && this.coefficients.get(this.coefficients.size() - 1)

```



```

165         .equal(zero)) {
166             counter++;
167             this.coefficients.remove(this.coefficients.size() - 1);
168         }
169         return counter;
170     }
171
172     @Override
173     public Polynomial sub(Polynomial p) {
174         Polynomial result = new Polynomial(zero);
175         // Subtract term by term
176         if (this.coefficients.size() < p.coefficients.size()) {
177             int i;
178             for (i = 0; i < this.coefficients.size(); i++) {
179                 result.coefficients.add(this.coefficients.get(i).sub(
180                     p.coefficients.get(i)));
181             }
182             for (; i < p.coefficients.size(); i++) {
183                 result.coefficients.add(p.coefficients.get(i).minus());
184             }
185         }
186         else {
187             int i;
188             for (i = 0; i < p.coefficients.size(); i++) {
189                 result.coefficients.add(this.coefficients.get(i).sub(
190                     p.coefficients.get(i)));
191             }
192             for (; i < this.coefficients.size(); i++) {
193                 result.coefficients.add(this.coefficients.get(i));
194             }
195         }
196         result.clean();
197         return result;
198     }
199
200     @Override
201     public Polynomial div(Polynomial p) {
202         // If the order of current polynomial is lower than that of p, return
203         // zero.
204         if (this.coefficients.size() < p.coefficients.size()) {
205             return zero();
206         }
207         Polynomial result = new Polynomial(zero);
208         // Pad the result with correct number of zeros for the coefficients
209         for (int i = 0; i < this.coefficients.size()
210             - p.coefficients.size() + 1; i++) {
211             result.coefficients.add(zero);
212         }
213         Polynomial temp = new Polynomial(this);
214         // Do the polynomial division
215         for (int j = temp.coefficients.size() - p.coefficients.size(); j >= 0;) {
216             // Calculate the coefficients when the highest order term in the
217             // current polynomial is divided by the highest order term in
218             // polynomial p.
219             FiniteField<Int> c =

```

```

220         temp.coefficients
221             .get(temp.coefficients.size() - 1)
222             .div(p.coefficients.get(p.coefficients.size() - 1));
223     result.coefficients.set(j, c);
224     // Multiply the coefficient c to p and subtract the result from temp
225     for (int i = 1; i <= p.coefficients.size(); i++) {
226         temp.coefficients
227             .set(temp.coefficients.size() - i,
228                 temp.coefficients.get(
229                     temp.coefficients.size() - i)
230                     .sub(p.coefficients
231                         .get(p.coefficients
232                             .size() - i).mul(
233                                 c)));
234     }
235     // Clean the high order terms in temp that has a zero coefficient
236     // and decrease j accordingly.
237     j -= temp.clean();
238 }
239 result.clean();
240 return result;
241 }
242
243 @Override
244 public Polynomial mod(Polynomial p) {
245     if (this.coefficients.size() < p.coefficients.size()) {
246         return this;
247     }
248     // See the notes for div() for the following code. They are exactly the
249     same except there is no need to create a polynomial to hold the
250     result.
251     Polynomial temp = new Polynomial(this);
252     for (int j = temp.coefficients.size() - p.coefficients.size(); j >= 0;) {
253         FiniteField<Int> c =
254             temp.coefficients
255                 .get(temp.coefficients.size() - 1)
256                 .div(p.coefficients.get(p.coefficients.size() - 1));
257         for (int i = 1; i <= p.coefficients.size(); i++) {
258             temp.coefficients
259                 .set(temp.coefficients.size() - i,
260                     temp.coefficients.get(
261                         temp.coefficients.size() - i)
262                         .sub(p.coefficients
263                             .get(p.coefficients
264                                 .size() - i).mul(
265                                     c)));
266         }
267         j -= temp.clean();
268     }
269     temp.clean();
270     return temp;
271 }
272
273 @Override
274 public Polynomial minus() {

```

```

275     Polynomial result = new Polynomial(zero);
276     // Inverse the coefficients term by term.
277     for (FiniteField<Int> c : this.coefficients) {
278         result.coefficients.add(c.minus());
279     }
280     return result;
281 }
282
283 @Override
284 public String toString() {
285     if (this.coefficients.isEmpty())
286         return "0";
287     StringBuilder sb = new StringBuilder();
288     for (int i = this.coefficients.size() - 1; i >= 0; i--) {
289         String coefficient;
290         if (this.coefficients.get(i).equal(zero))
291             continue;
292         if (this.coefficients.get(i).equal(zero.one()) && i != 0) {
293             coefficient = "";
294         }
295         else {
296             coefficient = "" + this.coefficients.get(i);
297         }
298         if (i == 0) {
299             sb.append(coefficient + " + ");
300             continue;
301         }
302         if (i == 1) {
303             sb.append(coefficient + "x + ");
304             continue;
305         }
306         sb.append(coefficient + "x^" + i + " + ");
307     }
308     return sb.substring(0, sb.length() - 3);
309 }
310
311 @Override
312 public void setValue(Polynomial p) {
313     this.zero = p.zero;
314     this.coefficients = p.coefficients;
315 }
316
317 }

```

Listing 12: PolyByte.java

```

1 import java.util.List;
2
3 /**
4  * PolyByte.java
5  * This class is a wrapper class of FiniteField on a Polynomial. It represents a
6  * byte in the polynomial form.
7  * It provides the conversion from a byte to a PolyByte and the basic arithmetic
8  * operations supported by a finite field on polynomials.
9  *
10 * @author Tianyu Geng, Brandon Amos
11 * @version Dec 2, 2012
12 */
13 public class PolyByte {
14     // The internal finite field of polynomial.
15     private FiniteField<Polynomial> value;
16     // The upper bound for the coefficients.
17     private static Int N;
18     // The zero of the coefficients of this polynomial.
19     private static FiniteField<Int> zero;
20     // The irreducible polynomial for the polynomial field.
21     private static Polynomial PN;
22     // Initialize the static variables.
23     {
24         N = new Int(2);
25         zero = new FiniteField<Int>(N);
26         PN = new Polynomial(zero);
27         //  $x^8 + x^4 + x^3 + x + 1$ 
28         PN.setCoefficients(1, 1, 0, 1, 1, 0, 0, 0, 1);
29     }
30
31     /**
32     * Construct a PolyByte basing on the internal value.
33     *
34     * @param value
35     *         the internal value
36     */
37     public PolyByte(FiniteField<Polynomial> value) {
38         this.value = value;
39     }
40
41     /**
42     * Construct a PolyByte basing on a integer byte.
43     *
44     * @param v
45     *         the integer byte
46     */
47     public PolyByte(int v) {
48         Polynomial p = new Polynomial(zero);
49         for (int i = 0; i < 8; i++) {
50             FiniteField<Int> c =
51                 new FiniteField<Int>(new Int((v >> i) % 2), N);
52             p.addHigherPower(c);
53         }
54         p.clean();

```

```

55     value = new FiniteField<Polynomial>(p, PN);
56 }
57
58 /**
59  * Convert this PolyByte to a integer byte.
60  *
61  * @return the byte
62  */
63 public int toByte() {
64     List<FiniteField<Int>> cs = value.getValue().getCoefficients();
65     int result = 0;
66     for (int i = 0; i < value.getValue().getCoefficients().size(); i++) {
67         result += cs.get(i).getValue().getValue() << i;
68     }
69     return result;
70 }
71
72 /**
73  * Find the inverse of this PolyByte
74  *
75  * @return the inverse of this PolyByte
76  */
77 public PolyByte inverse() {
78     // in the case with zero, just return zero
79     if (value.equal(value.zero())) {
80         return new PolyByte(value.zero());
81     }
82     return new PolyByte(value.inverse());
83 }
84
85 /**
86  * Multiply this PolyByte with another PolyByte and return the result.
87  *
88  * @param pb
89  *         the other PolyByte
90  * @return the result
91  */
92 public PolyByte mul(PolyByte pb) {
93     return new PolyByte(value.mul(pb.value));
94 }
95
96 /**
97  * Add this PolyByte with another PolyByte and return the result.
98  *
99  * @param pb
100         the other PolyByte
101  * @return the result
102  */
103 public PolyByte add(PolyByte pb) {
104     return new PolyByte(value.add(pb.value));
105 }
106
107 /**
108  * Subtract this PolyByte by another PolyByte and return the result.
109  *

```

```

110     * @param pb
111     *           the other PolyByte
112     * @return the result
113     */
114     public PolyByte sub(PolyByte pb) {
115         return new PolyByte(value.sub(pb.value));
116     }
117
118     /**
119     * Divide this PolyByte by another PolyByte and return the result.
120     *
121     * @param pb
122     *           the other PolyByte
123     * @return the result
124     */
125     public PolyByte div(PolyByte pb) {
126         return new PolyByte(value.div(pb.value));
127     }
128
129     /**
130     * Raise the current PolyByte to the nth degree
131     *
132     * @param n
133     *           the power
134     * @return the result
135     */
136     public PolyByte pow(int n) {
137         FiniteField<Polynomial> rvalue = value.one();
138         for (int i = 0; i < n; i++) {
139             rvalue = rvalue.mul(value);
140         }
141         return new PolyByte(rvalue);
142     }
143
144     @Override
145     public String toString() {
146         return value.toString();
147     }
148
149 }

```